Personal Finance Manager

Iteration 1 — Written Deliverable

Course: CSE 3311
Team: Team 6
Team Members: Ujjwal Sood, Hardik Kari, David Aviles, Ishika Sanghadia
Instructor: Christoph Csallner

---

1. Scope and Project Plan

Two connected features were implemented on this first iteration: a basic income/expense log and a one-time purchase affordability check. Both use one preset account. The UI displays the balance, fixed bill and savings reserves, and the calculation behind a purchase result. This first increment lets us evaluate the core workflow before expanding the application.

An updated project plan for future iterations is shown below:

Iteration	Priority and planned features
1	Must have: transaction entry, shared balance, purchase check, validation, tests, and run instructions. Implemented.
2	Proposed should have: editable commitments, transaction corrections, and persistence. Done when changes update the calculator correctly, data survives restart, and users can explain the revised result.
3	Proposed could have: recurring schedules, savings goals, and simple reports, selected from feedback. Implement authentication for multi-user deployment. Preserve time for regression tests and final demo.

2. Competitor Re-evaluation

YNAB emphasizes giving available money a purpose and planning for future or irregular expenses. Our reserve model shares that principle but presents an individual purchase and its remaining funds in one short flow. [1]

After reinvestigating Rocket Money, it also tracks spending and describes Safe to Spend as funds remaining after relevant obligations before the next payday. This directly overlaps our concept, so we do cannot claim affordability guidance is unique. However, Rocket Money is centered around managing subscriptions which may not apply to many customers in our user base. [2]

Monarch supports category and flex budgeting around expected monthly cash flow. Our first iteration instead evaluates recorded cash against two fixed reserves. It is simpler, but less complete and less automated. [3]

Our proposed advantage is an understandable purchase-specific calculation connected to a small manual log, reachable without account setup in the demo. Whether users prefer it is still a hypothesis. The comparison supports testing clarity and usefulness rather than assuming a competitive advantage.

3. Risk Re-assessment

The five newly re-evaluated risks below are ranked by probability multiplied by impact in additional team-hours of rework.

Risk	**Probability x Hours**	**Exposure**	**Mitigation**
Scope Creep	60% x 15	9.0 Hours	Now: keep the two-feature baseline. Iteration 2 planning/midpoint: cut lower priorities if core work slips.
Users misunderstand or do not value the check	50% x 16	8.0 Hours	Now: show assumptions and arithmetic. Before Iteration 2: observe users and retest revised wording.
Incorrect currency or reserve logic	35% x 12	4.2 Hours	Now: Decimal validation and boundary tests. Every change: regressions. Before recurring bills: test payment matching.
Data is stale lost or misleading	40% x 8	3.2 Hours	Now: synthetic data, labeled fixed reserves, separate test copies. Iteration 2: persistence and commitment updates.
Setup blocks independent testing	30% x 8	2.4 Hours	Before distribution: verify clean uv setup on another machine and generate a lockfile. Compiled frontend already included.

---

4. User Feedback

Our proposed initial users are college students and young professionals who manually track everyday spending and want to understand whether a discretionary purchase fits after commitments. The key needs to validate are quick entry, a clear explanation of spendable money, and confidence that the result reflects relevant obligations.

Proposed sessions: observe target users checking $60 and $500 purchases, logging a $60 expense, and explaining why the $500 result changes. Ask how they currently decide what they can afford, what was difficult, and which missing information matters.

For this first iteration, we had one session with a University of Texas at Arlington student who is majoring in Computer Science. They were asked to complete different actions: check purchases and budgets, log expenses, and check how a purchase affects finances. The user was able to successfully complete the actions along with providing feedback over the demo.

Concern/Comment	Explanation
Display of Reserves	The user would like to have the reserves broken down into different boxes to quickly differentiate between them.
Display of the Spending Log	The user would like to have the transaction log moved further up by changing the transaction entry to a pop-up dialog with an “Add Transaction” button
Display of the overall financial summaries	The user visited the demo page on a mobile device and would like to have the financial summary boxes smaller, so they don’t occupy a substantial amount of screen space.

Based on this session, we immediately implemented the user's feedback regarding screen space by adjusting the CSS to reduce the size of the financial summary boxes for Iteration 1. The larger structural requests, such as the pop-up transaction dialog and breaking down the display of reserves, have been added to our project plan to be implemented in Iteration 2. Moving forward, we will start user testing earlier in the iteration to ensure we can capture and act on feedback sooner, along with finding more users to provide feedback.

---

5. Specification and Design

React collects inputs and displays the server’s results. FastAPI validates request shapes. DemoStore manages one account and ledger and calls the domain calculator; the UI does not implement a second financial formula.

Key Structure	Fields and Responsibility
Account	name: string; starting_balance: Decimal. Computes opening balance plus income minus expenses.
Transaction	Frozen amount: Decimal; category: string; transaction_type: income/expense; transaction_date: date; account_id: 1; note: string. Validates completed entries.
DemoStore	Demo account; transaction list; upcoming_required and savings_remaining: Decimal; reentrant lock. Supports snapshot, add, check, and reset consistently.
AffordabilityResult	Decimal fields: purchase_amount, current_available, upcoming_required, savings_remaining, projected_remaining. Also can_afford: Boolean and message: string.
UI State	Selected screen, form values, snapshot, pending request, error/notice, and result. Optional item label is UI-only.

Rules and Starting data

```
balance = opening balance + logged income - logged expenses
available = balance - bill reserve - savings reserve
remaining = available - purchase price
can_afford = remaining >= 0
```

Opening balance of $1,000 plus $500 income minus $75 groceries and $25 transport gives $1,400. Fixed reserves are $800 for unpaid bills and $100 for additional savings, leaving $500 available. Seed dates use the current date and the preceding two days. A check is hypothetical and never adds a transaction. Zero remaining is an exact fit with no extra unreserved money.

Already logged expenses reduce the balance once. The bill reserve represents still-unpaid commitments, but this prototype does not release a reserve when its bill is paid. Use discretionary transactions in the demo; automatic bill matching is later work. Valid expenses may overdraw the account because the log records actual completed spending.

Inputs and Validation

Input	Constraint
Amount	Required API decimal string, maximum 32 characters; finite, whole cents, $0.01 to $1,000,000.00. UI uses numeric fields; backend rejects fractional cents rather than rounding.
Type and Category	Income or expense; category 1-40 characters and nonblank after trimming. Account ID is fixed and cannot be supplied in requests.
Date and Note	Valid ISO date, today or earlier; optional note up to 160 characters. Category/note trimmed. UI defaults to local today; backend validates against server today.
Purchase Item	Optional UI-only label up to 80 characters. Purchase request sends only amount. Unknown transaction/purchase API fields are rejected.

The ledger holds at most 1,000 entries including seeds. Newest-entered entries appear first. Each server process owns one shared account; run one worker. Browser reloads preserve server data, while restart or reset restores seeds. Separate users need separate local copies for independent state.

---

6. Outputs and Use Cases

API	Inputs and Outputs
GET /api/demo	No input. 200 snapshot: name, starting_balance, current_balance, upcoming_required, savings_remaining, spendable, transactions.
POST /api/transactions	amount, category, transaction_type, transaction_date, optional note. 201 updated snapshot; 422 on invalid input without mutation.

POST /api/affordability	amount. 200 AffordabilityResult fields in Section 3; 422 on invalid input. No ledger change.
POST /api/reset	No required body. 200 seeded snapshot; clears the current ledger and restores the three examples.

Every currency output is a JSON string with two decimal places. A transaction output contains amount, category, transaction_type, transaction_date, account_id, and note. UI outputs include the account summary, transaction list, calculation breakdown, result text, validation messages, loading status, and success/failure notices.

UC1: Check a purchase

Actor: demo user. Precondition: account loaded. Flow: open Can I afford it, enter optional item/price, and select Check purchase. The server uses the current ledger and reserves; the UI displays each subtraction and the result. Postcondition: result displayed; ledger unchanged.

Alternatives: positive remainder fits; zero fits exactly and warns that no extra money remains; negative does not fit. Missing, zero, negative, non-finite, fractional-cent, or over-limit amounts are rejected. A request failure shows an error and permits retry. Changing item/price clears the old result; logging a transaction or resetting also invalidates it.

UC2: Log income or expense

Actor: demo user. Precondition: account loaded. Flow: open Spending log, select type, enter amount/category/date and optional note, then save. The validated entry is added and the list/balance update. Postcondition: the next purchase check reflects the same account's new balance.

Alternatives: invalid amount, blank/overlong category, unsupported type, invalid/future date, overlong note, extra account field, or full log rejects the entry. Failed requests preserve inputs. Valid overdraft spending is recorded. Controls disable during submission. If the connection fails after a possible save, reload and inspect the log before retrying; duplicate detection is not implemented.

UC3: Navigation and Reset

The app opens on the calculator. Navigation connects Calculator and Spending log directly. Failed initial load shows Retry. Reset demo opens confirmation: cancel preserves state; confirm restores seeds and clears forms/results. Failed reset shows an error. Restart also restores seeds; reload only refreshes UI state and the server snapshot.

Another browser tab may change the shared server account. Reload to refresh a stale summary; each new check uses the current server balance. This demo has no live synchronization,

authentication, persistent storage, transaction editing, or scheduled transactions. These limitations are explicit dependencies for later iterations.

Demonstration Example

With $500 available, checking $500 returns $0 and an exact-fit message. Logging a $60 expense reduces the balance to $1,340 and available funds to $440. Checking $500 again returns -$60. This demonstrates the connection between the log and the calculator without introducing additional features.

---

7. Code Tests and Setup

Domain classes isolate financial rules; DemoStore holds application state; main.py handles HTTP; React handles the UI. Comments explain fractional-cent rejection, avoiding double-counting, overdraft recording, validation before mutation, and decimal-string serialization.

Verification	Evidence and Result
Domain tests	Purchase boundaries $60/$500/$500.01; zero/negative balance; invalid amounts; decimal precision; income/expense signs; transaction validation; negative reserves; check/reset state rules.
API tests	Full seed/check/save/income/reset flow; invalid requests preserve data; string amount requirement; invalid/future dates; extra fields; overdraft logging; removed routes absent.
Observed result	46 parameterized tests passed on Python 3.12. Fresh app/store per API test.
Build and browser	TypeScript/Vite build passed. Browser verified seed values, exact fit, expense save, updated balance and rejection, and reset. These are developer checks, not recruited-user feedback.

Not yet verified: exhaustive mobile/cross-browser behavior, load testing, the full-log boundary, and duplicate-retry recovery. The uv tool manages project dependencies and the python version needed to ensure execution on other systems.

Running and Testing from the project root

Clone the GitHub project repository. Install uv if needed. The backend pins Python 3.12 and declares runtime/dev dependencies; uv creates backend/.venv automatically. The compiled frontend is included, so Node is unnecessary just to try the demo. First setup needs internet access for missing Python/packages. [4]

```
uv sync --directory backend
uv run --directory backend uvicorn app.main:app --port 8000
# In another terminal, run the test suite
uv run --directory backend pytest -q
```

Open http://127.0.0.1:8000 and keep the server terminal running; Ctrl+C stops it. Commands work on Windows, macOS, and Linux. If port 8000 is occupied, use --port 8001 and open that URL. If inside the backend folder, omit --directory backend.

Build changes and repeat the demo

Frontend rebuild requires Node.js 22.12+ and pnpm 11.25.0. From the root, run cd frontend, pnpm install --frozen-lockfile, then pnpm run build. Restart the backend and reload. For live edits, use pnpm dev with the backend on port 8000 and open port 5173. If the main page is missing, check that frontend/dist was included or rebuild it.

Manual acceptance: reset; check $60 (leaves $440), $500 ($0), and $500.01 (-$0.01); log $60 expense (balance $1,340); recheck $500 (-$60); add $100 income (balance $1,440); try invalid inputs; reload and reset. Verify no unintended entry was added by a purchase check.

References

[1] YNAB

The YNAB Method. Official methodology page. https://www.ynab.com/ynab-method

[2] Rocket Money

Clay Conger, Tracking expenses with Rocket Money, May 5 2026. Includes Safe to Spend. https://www.rocketmoney.com/learn/personal-finance/tracking-expenses-with-rocket-money

[3] Monarch

Creating Your Budget in Monarch. Official help article on category/flex budgeting and monthly cash flow. https://help.monarchmoney.com/hc/en-us/articles/360048883631-Budgets

[4] Project GitHub Repository – Iteration 1

https://github.com/soodujjwal/Team_6_Object_Oriented_Software/tree/iteration-1

Generative AI (Codex) was used in this project to assist with code development and documentation drafting/creation